Understanding NumPy's ndarray

The ndarray (N-dimensional array) is the core foundational object in NumPy. It is a fast, flexible, and memory-efficient container for large datasets of items of the same type.

Key Features

- **Homogeneous Data:** All elements must be the same data type
- **Fixed Size:** Unlike Python lists, size is fixed when created
- **Vectorized Operations:** Perform math operations on entire arrays without loops

---

How to Create an ndarray

1D array example

```
import numpy as np

arr1 = np.array([1, 2, 3, 4])
```

2D array example

```
import numpy as np

arr2 = np.array([[1, 2, 3], [4, 5, 6]])
```

Important Attributes

Attribute	Meaning	Example
.ndim	Number of dimensions	1 for a 1D array, 2 for a 2D array
.shape	Size in each dimension	(4,) or (2, 3)
.size	Total number of elements	4 or 6
.dtype	Data type	int64, float32, etc.

Example: Using These Attributes

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.ndim)  # 2
print(arr.shape)  # (2, 3)
print(arr.size)  # 6
print(arr.dtype)  # int64
```

Why Use ndarray?

- **Speed:** Implemented in C, much faster than Python loops
- **Memory Efficiency:** Takes less memory than Python lists

Indexing (Accessing a Single Item)

Indexing allows you to grab a specific, single element using its position index. Python always starts counting from 0. For 2D arrays (matrices), you specify [row_index, column_index].

Code example:

```
import numpy as np

matrix = np.array([
   [10,20,30],
   [40,50,60]
   ])

print("Row 0,Col 0: ", matrix[0,0])
print("Row 0,Col 1: ", matrix[0,1])
print("Row 0,Col 2: ", matrix[0,2])
print("Row 1,Col 0: ", matrix[1,0])
print("Row 1,Col 1: ", matrix[1,1])
print("Row 1,Col 2: ", matrix[1,2])

print(matrix)
```

Slicing (Extracting a Range or Sub-section)

Slicing lets you extract a portion (range) of an array using the syntax [start:stop]. The stop index is exclusive (it goes up to, but doesn't include, that number). For 2D arrays, you slice rows and columns separated by a comma: [row_start:row_end, col_start:col_end].

Code example:

```
import numpy as np

arr = np.array([10,20,30,40,50])

print("Slice [1:4]: ", arr[1:4])

mat = np.array([
   [1,2,3],
   [4,5,6],
   [7,8,9]
   ])

print("Top-left 2x2:\n", mat[:2,:2])
```

Fancy Indexing (Picking Arbitrary Positions)

Fancy indexing allows you to pass a list of indices instead of a single number or a continuous slice. This lets you pull out rows or columns in any custom order you want.

Code example:

```
import numpy as np

matrix = np.array([
   [10,20,30],
   [40,50,60],
   [70,80,90],
])

print("Reordered rows: \n", matrix[[2,1,0]])
```

Boolean Masking (Filtering by Condition)

Explanation: Boolean masking lets you filter data using a condition (like `arr > 5`). NumPy creates a True/False "mask" behind the scenes and returns *only* the elements where the condition is `True`.

```
import numpy as np

matrix = np.array([
   [10,20,30],
   [40,50,60],
   [70,80,90],
])

print("Reordered rows: \n", matrix[[2,1,0]])
```

Array Creation

`np.array()` — Creating from Python Lists

Explanation: This is the most basic way to create an array by passing a standard Python list or tuple into NumPy.

Example:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print("np.array:", arr)
```

np.zeros() — Creating an Array of Zeros

Explanation: Creates an array completely filled with 0.0. You pass a tuple representing the shape (rows, columns). It is very common in machine learning for initializing weights or placeholders.

```
# Create a 2x3 matrix of zeros
z = np.zeros((2, 3))
print("np.zeros:\n", z)
```

np.ones() — Creating an Array of Ones

Explanation: Similar to np.zeros(), but fills the array with 1.0. Useful when you need identity or scaling operations.

```
# Create a 3x2 matrix of ones
o = np.ones((3, 2))
print("np.ones:\n", o)
```

np.empty() — Creating an Uninitialized Array

Explanation: Creates an array without initializing its values. It allocates memory quickly and fills it with whatever random "garbage" data happens to be sitting in your computer's RAM at that moment. It's used when performance is critical and you plan to overwrite every value immediately.

```
# Create a 2x2 empty array
emp = np.empty((2, 2))
print("np.empty (random memory values):\n", emp)
```

np.arange() — Creating Sequences with a Step

Explanation: Works just like Python's built-in range() function, but returns a NumPy array. You specify (start, stop, step).

```
# Numbers from 0 up to 10, stepping by 2
seq = np.arange(0, 10, 2)
print("np.arange:", seq)  # Output: [0 2 4 6 8]
```

np.linspace() — Creating Linearly Spaced Numbers

Explanation: Generates a specific quantity of numbers evenly spaced between a start and stop point. You specify (start, stop, total_elements). This is heavily used in plotting graphs and math functions.

```
# 5 numbers evenly spaced between 0 and 1
space = np.linspace(0, 1, 5)
print("np.linspace:", space)  # Output: [0.   0.25 0.5
0.75 1.  ]
```

np.eye() — Creating an Identity Matrix

Explanation: Creates a square matrix where all the elements on the main diagonal are 1 and all other elements are 0. Essential for linear algebra.

```
# Create a 3x3 identity matrix
identity = np.eye(3)
print("np.eye (Identity Matrix):\n", identity)
```

np.random — Creating Random Arrays

Explanation: The np.random module generates arrays filled with random numbers. Common functions include:

np.random.rand(d0, d1): Random floats between 0 and 1 (Uniform distribution).

np.random.randint(low, high, size): Random integers within a specific range.

```
# 3 random floats between 0 and 1
rand_floats = np.random.rand(3)
print("Random floats:", rand_floats)

# A 2x2 matrix of random integers between 1 and 10
rand_ints = np.random.randint(1, 11, size=(2, 2))
print("Random integers:\n", rand_ints)
```

Shape Manipulation

reshape() — Changing Array Dimensions

Explanation: reshape() changes the shape of an array without altering its data. The total number of elements must remain the same (e.g., you cannot reshape a 6-element array into a 2x4 matrix because that requires 8 elements).

```
import numpy as np

# Create a 1D array with 6 numbers
arr = np.arange(6)
print("Original 1D array:", arr)

# Reshape it into a 2 rows by 3 columns matrix
matrix = arr.reshape(2, 3)
print("Reshaped to 2x3:\n", matrix)
```

flatten() — Collapsing into 1D (Copy)

Explanation: flatten() collapses any multi-dimensional array into a 1D array. Crucially, it always returns a **copy** of the data—meaning if you change values in the flattened array, the original array remains untouched.

```
import numpy as np

matrix = np.array([[1, 2], [3, 4]])
flat_arr = matrix.flatten()

print("Flattened array:", flat_arr)  # Output: [1 2 3 4]
```

ravel() — Collapsing into 1D (View)

Explanation: Similar to flatten(), ravel() flattens a multi-dimensional array into 1D. However, instead of making a copy, it returns a view (sharing the same memory location if possible), making it faster and more memory-efficient.

```
import numpy as np

matrix = np.array([[1, 2], [3, 4]])
rav_arr = matrix.ravel()

print("Raveled array:", rav_arr)  # Output: [1 2 3 4]
```

transpose() (or .T) — Flipping Axes

Explanation: Transposing flips the axes of an array. For a 2D matrix, it swaps rows and columns (turning row 0 into column 0, and column 0 into row 0). You can use .transpose() or the quick shorthand .T.

```
import numpy as np

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print("Original shape (2,3):\n", matrix)

# Transpose the matrix
transposed = matrix.T  # or matrix.transpose()
print("Transposed shape (3,2):\n", transposed)
```

swapaxes() — Interchanging Two Specific Axes

Explanation: While transpose() flips all axes, swapaxes() allows you to specifically choose **two axes** to interchange by passing their index numbers.

```
import numpy as np

# A 3D array layout placeholder
arr_3d = np.zeros((2, 3, 4))
print("Original shape:", arr_3d.shape)  # Output: (2, 3, 4)

# Swap axis 0 and axis 1
swapped = arr_3d.swapaxes(0, 1)
print("Swapped shape:", swapped.shape)  # Output: (3, 2, 4)
```

expand_dims() — Adding a New Dimension

Explanation: Inserts a new axis (dimension of size 1) at a specified position. This is heavily used in Deep Learning when you need to add a "batch size" dimension to an image or text array.

```
import numpy as np

arr = np.array([1, 2, 3])
print("Original shape:", arr.shape)  # Output: (3,)

# Add a new dimension along axis 0 (making it a row vector)
expanded = np.expand_dims(arr, axis=0)
print("Expanded shape:", expanded.shape)  # Output: (1, 3)
print("Expanded array:\n", expanded)
```

squeeze() — Removing Size-1 Dimensions

Explanation: Removes any dimensions from the array shape that have a length of 1. It is the exact opposite of expand_dims(), used to clean up unnecessary single-item dimensions.

```
import numpy as np

# An array with extra dimensions of size 1
arr_ones = np.array([[[1, 2, 3]]])
print("Original shape:", arr_ones.shape)  # Output: (1, 1, 3)

# Remove all dimensions of size 1
squeezed = arr_ones.squeeze()
print("Squeezed shape:", squeezed.shape)  # Output: (3,)
print("Squeezed array:", squeezed)
```

Numerical Computation

Vectorization

Explanation: Vectorization is the process of executing operations on entire arrays simultaneously rather than processing elements one by one. NumPy achieves this by delegating the heavy lifting to highly optimized C code under the hood, making calculations drastically faster than standard Python code.

```
import numpy as np

arr = np.array([1, 2, 3, 4])

# Vectorized operation: multiplies every element by 2 instantly without a loop
result = arr * 2
print("Vectorized Result:", result)  # Output: [ 2  4  6  8]
```

Avoid Python Loops

Explanation: Native Python for loops are slow because Python is dynamically typed and uses an interpreter that checks types at every iteration. By replacing loops with built-in NumPy vectorized functions or operators, you let NumPy execute loops in compiled C code.

```
import numpy as np

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

# Efficient: Adding arrays directly instead of using a Python 'for' loop
total = a + b
print("Without Loops:", total)  # Output: [11 22 33]
```

Memory Efficiency

Explanation: Standard Python lists are arrays of pointers scattered across random locations in RAM, which wastes memory and slows down CPU cache access. NumPy arrays are contiguous memory blocks —meaning all elements of the same data type are stored right next to each other in RAM, allowing for rapid CPU processing.

```
import numpy as np

# Create a large array of integers and check its total memory footprint in bytes
arr = np.arange(1000, dtype=np.int32)
print("Total bytes used in RAM:", arr.nbytes)
```

Views vs. Copies

Explanation:

View: A view does not own its data; it points to the memory of the original array. Modifying a view will modify the original array, but it saves memory and time because no new data is duplicated.

Copy: A copy creates a completely new array in a separate memory location. Modifying the copy leaves the original array untouched.

```
import numpy as np

original = np.array([1, 2, 3, 4])

# Slicing typically creates a view
view_arr = original[1:3]
view_arr[0] = 99  # Modifying the view

print("Original Array is affected:", original)  # Output: [ 1 99  3  4]
```

Dtype Optimization

Explanation: NumPy defaults to 64-bit data types (int64, float64), which take up more memory. If your data values fit in smaller ranges (such as image pixel values ranging from 0 to 255), optimizing your data type to int8 or float32 cuts your memory usage in half or quarters.

```
import numpy as np

# Using int8 to optimize memory for small integer values (-128 to 127)
optimized_arr = np.array([10, 20, 30], dtype=np.int8)
print("Optimized Data Type:", optimized_arr.dtype)
# Output: int8
print("Bytes used:", optimized_arr.nbytes)
```

Broadcasting Efficiency

Explanation: Broadcasting allows NumPy to perform arithmetic operations on arrays of different shapes. Instead of physically duplicating data to match shapes (which would waste memory), NumPy treats the smaller array as if it has the matching shape during the mathematical operation.

```
import numpy as np

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

scalar_row = np.array([10, 20, 30])

# Broadcasting adds the 1D row to every row of the 2D matrix automatically
result = matrix + scalar_row
print("Broadcasted Result:\n", result)
# Output:
# [[11 22 33]
#  [14 25 36]]
```

BroadCusting

Shape Compatibility & Broadcast Rules

Explanation: NumPy allows arithmetic operations between arrays of different shapes through Broadcasting. However, the shapes must be compatible.

The Rule: To determine if broadcasting is valid, NumPy compares the array shapes from right to left (trailing dimensions). Two dimensions are compatible if:

They are equal, or

One of them is 1.

If neither condition is met, NumPy throws a ValueError: operands could not be broadcast together.

Scalar Operation

Explanation: The simplest form of broadcasting is between an array and a scalar (a single number). NumPy treats the scalar as if it were an array of the same shape as the other array by conceptually stretching (replicating) it.

```
import numpy as np

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

# Scalar 10 is broadcasted to match the 2x3 shape
result = matrix + 10
print("Scalar Operation Result:\n", result)
# Output:
# [[11 12 13]
#  [14 15 16]]
```

Vector Operation (Element-wise)

Explanation: When operating on two arrays of the same shape, NumPy performs element-wise operations instantly without any loops (vectorization).

Row and Column Broadcasting

When broadcasting a 1D array (vector) with a 2D array (matrix), the direction matters based on their shapes:

Row Broadcast (1D to 2D)

Explanation: If a 1D array has a shape of (3,) and is added to a 2D matrix of shape (2, 3), the 1D array is automatically broadcasted across all rows of the matrix because the trailing dimensions match (3 == 3).

```
import numpy as np

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])  # Shape: (2, 3)

row_vector = np.array([10, 20, 30])  # Shape: (3,)

# Row broadcast: adds row_vector to every row of the matrix
result = matrix + row_vector
print("Row Broadcast Result:\n", result)
# Output:
# [[11 22 33]
#  [14 25 36]]
```

Column Broadcast (1D to 2D)

Explanation: If you want to add a 1D vector across the columns of a matrix, its shape must have a 1 in the column dimension (e.g., shape (2, 1)), making it compatible with a (2, 3) matrix. You can use expand_dims() or slicing to achieve this.

```
import numpy as np

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])  # Shape: (2, 3)

# Reshape vector to shape (2, 1) to broadcast vertically across columns
col_vector = np.array([[10],
                       [20]])  # Shape: (2, 1)

result = matrix + col_vector
print("Column Broadcast Result:\n", result)
# Output:
# [[11 12 13]
#  [24 25 26]]
```

Data Processing

Concatenation

Explanation: Concatenation joins two or more arrays along an existing axis. For 1D arrays, it appends them end-to-end. For 2D arrays, you can join them row-wise (axis=0) or column-wise (axis=1).

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Join two 1D arrays
result = np.concatenate((a, b))
print("Concatenated:", result)  # Output: [1 2 3 4 5 6]
```

Stack (`np.stack`)

Explanation: Unlike concatenation (which uses an *existing* axis), stacking joins arrays along a **new axis**, increasing the dimensions of the resulting array by 1. (Variants include `np.vstack` for vertical stacking and `np.hstack` for horizontal stacking).

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Stack along a new axis (axis=0)
stacked = np.stack((a, b), axis=0)
print("Stacked Matrix:\n", stacked)
# Output:
# [[1 2 3]
#  [4 5 6]] (Shape becomes 2x3)
```

Split (np.split)

Explanation: Splitting is the exact opposite of concatenation. It breaks a single array into multiple sub-arrays at specified index points or into equal sections.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])

# Split the array into 3 equal sub-arrays
sub_arrays = np.split(arr, 3)
print("Split results:", sub_arrays)
# Output: [array([10, 20]), array([30, 40]), array([50, 60])]
```

Sort (np.sort)

Explanation: Returns a sorted copy of an array along a specified axis. By default, it sorts in ascending order.

```
import numpy as np

arr = np.array([31, 5, 12, 8])

# Sort elements in ascending order
sorted_arr = np.sort(arr)
print("Sorted Array:", sorted_arr)  # Output: [ 5  8 12 31]
```

Unique (np.unique)

Explanation: Finds all unique elements in an array by stripping away duplicates and returning the sorted unique values. It is very useful for categorical data or labels in machine learning.

```
import numpy as np

data = np.array([1, 2, 2, 3, 1, 4, 5, 5])

unique_values = np.unique(data)
print("Unique Elements:", unique_values)  # Output: [1 2 3 4 5]
```

Where (np.where)

Explanation: Returns the indices where a given conditional statement is true. It can also be used for conditional replacement (`np.where(condition, value_if_true, value_if_false)`).

```
import numpy as np

arr = np.array([10, 15, 20, 25, 30])

# Find indices where values are greater than 20
indices = np.where(arr > 20)
print("Indices where > 20:", indices)  # Output: (array([3, 4]),)

# Conditional replacement: Replace values > 20 with 1, else 0
modified = np.where(arr > 20, 1, 0)
print("Modified:", modified)  # Output: [0 0 0 1 1]
```

NaN Handling (Not a Number)

Explanation: In real-world datasets, missing values are often represented as `NaN`. `np.isnan()` checks where missing values exist, and `np.nan_to_num()` safely replaces `NaN` values with zeros (or other numbers) so models don't crash.

```
import numpy as np

data = np.array([1.0, 2.5, np.nan, 4.1, np.nan])

# Check which elements are NaN
print("Is NaN:", np.isnan(data))  # [False False True False  True]

# Clean data by replacing NaN with 0.0
cleaned_data = np.nan_to_num(data, nan=0.0)
print("Cleaned Data:", cleaned_data)  # [1.  2.5 0.  4.1 0. ]
```

Inf Handling (Infinity)

Explanation: Mathematical operations like dividing a number by zero result in infinity (inf or -inf). np.isinf() detects them, and np.nan_to_num() can convert infinite values into large finite numbers or zeros.

```
import numpy as np

arr = np.array([10.0, 5.0, np.inf, -np.inf])

# Check for infinity values
print("Is Inf:", np.isinf(arr))  # [False False  True  True]

# Replace inf with large finite numbers or zeros
safe_arr = np.nan_to_num(arr, posinf=9999, neginf=-9999)
print("Safe Array:", safe_arr)  # [  10.    5.  9999. -9999.]
```